Yuta Hiruma

November 3, 2026

User Manual for the SubsApp

Contents

1. Introduction **2**

2. Installation **3**
2.1. Model code . . . 3
2.2. Repository structure . . . 3
2.3. Software requirements . . . 3
2.4. SWAP installation (Optional) . . . 4

3. Usage Guide **5**
3.1. Input data . . . 5
3.1.1. General input . . . 5
3.1.2. Soil material parameters . . . 6
3.2. Initial and boundary conditions . . . 8
3.3. Model run . . . 9
3.3.1. Main script . . . 9
3.3.2. Makefile and SLURM script . . . 9
3.4. Output . . . 10

A. SWAP model **11**
A.1. SWAP directory structure . . . 11
A.2. Input data . . . 11
A.2.1. Groundwater level . . . 12
A.2.2. Meteorological condition . . . 12
A.3. SWAP model run . . . 13
A.4. Conversion of SWAP output to SubsApp input . . . 13

1. Introduction

SubsApp is Python code to simulate the deformation of soft soils, particularly soft soils such as clay and peat, under both saturated and unsaturated conditions in one-dimention. The model represents soil deformation processes resulting from hydraulic changes, including consolidation, shrinkage, and swelling. Deformation is calculated for the soil profile extending from the ground surface to the top of a stable underlying layer, such as sand. This manual describes how to use SubsApp to simulate land subsidence driven by pore-pressure changes and hydraulic dynamics, primarily within Holocene deposits. It provides guidance on preparing input data, configuring model settings, running simulations, and interpreting model outputs. A comprehensive description of the model concepts, physical processes, governing equations, and numerical implementation is provided in the scientific publication (Ref: XXX). The current version of SubsApp focuses on deformation caused by hydro-mechanical processes. Other subsidence mechanisms, such as peat oxidation, organic matter decomposition, and settlement due to external loading, are not included.

2. Installation

2.1. Model code

The SubsApp source code is available from the GitHub repository:

`https://github.com/Yuta-hiruma/SubsApp.git`

To download the repository, execute the following commands in a terminal:

```
git clone https://github.com/Yuta-hiruma/SubsApp.git
cd SubsApp
```

2.2. Repository structure

The repository is organized as shown in Figure **??** . The main source code is located in the `src` directory. Input data, including soil parameters and boundary conditions, are stored in the `data` directory. Simulation outputs are written to the `results` directory and include figures (`.png`) and numerical output files (`.dat`). Benchmark cases and associated input data are provided in the `benchmarking` directory. A detailed description of all files and directories is provided in Section 3.

```
SubsApp/
 data/ # Input files and soil parameters
 doc/ # Documentation
 results/ # Simulation results
 src/ # Source code
              input/ # Initial and boundary condition
 benchmarking/ # Benchmark cases and input data
 README.md
 requirements.txt
```

Figure 1: *Directory structure of SubsApp on Git.*

2.3. Software requirements

SubsApp is developed in Python and has been tested with Python version 3.13.11. Python can be downloaded from: `https://www.python.org/`

The software has been tested with the Python packages listed in Table 1.

Table 1: *Tested Python packages.*

Package	Version
geopandas	0.14.4
matplotlib	3.7.2
numba	0.61.2
numba_cpython -m pip show numpy scipy matplotlib pandas numba daskelltree	0.4.1
numpy	1.26.4
numpydoc	1.5.0
pandas	2.3.1
scipy	1.11.1
dask	2026.3.0

The required packages can be installed manually. Alternatively, all required packages can be installed from the provided `requirements.txt` file:

```
pip install -r requirements.txt
```

GMT version 6.6.0 is used for data visualization.

2.4. SWAP installation (Optional)

SubsApp requires pore-pressure boundary conditions at the soil surface. These boundary conditions can be generated using the Soil-Water-Atmosphere-Plant (SWAP) model (Kroes et al. [2017]), which simulates variably saturated water flow based on Richards' equation. SWAP can be obtained from: `https://swap.wur.nl/`. Compilation of SWAP requires a Fortran compiler. The model has been tested using: GNU Fortran (`gfortran`) version 7.2.0.
Information on GNU Fortran is available at: `https://gcc.gnu.org/fortran/`

3. Usage Guide

3.1. Input data

Before running the model, soil geometries, simulation time, initial conditions, boundary conditions, and soil properties must be specified. All input files are located in the `data` directory.

3.1.1. General input

General simulation settings are defined in the file `input.py`. These settings include the site name, simulation period, initial groundwater level, reference elevation, and the geometry of the subsurface soil column. The model is primarily intended for simulations at timescales ranging from approximately one to ten years. Figure 2 shows an example of the general input settings defined in `input.py`. And the variables shown in Figure 2 are described in Table 2 :

```
# ------------------------------------------
# General Input
# ------------------------------------------
site = "small_strain"
tstart = "2024-01-01"
tend = "2025-01-01"

# ------------------------------------------
# Groundwater
# ------------------------------------------
gwli = 0 # Initial groundwater level [m]
Ref_Time = "10:00" # Reference time
GL_to_NAP = -1.48 # Ground-surface elevation [m NAP]

# ------------------------------------------
# Soil depths and types
# ------------------------------------------
depths = [-2.7, -5.0, -5.70, -6.0, -6.50]
types = ['Clay_top', 'Peat1', 'Clay2', 'Peat2', 'Sand']

# ------------------------------------------
# Boundary condition type at Top
# ------------------------------------------
#0 is Dirichlet boundary condition. (Pressure [kPa])
#1 is Neumann boundary condition (flux [m/day])
tp_bc = 1 #Top boundary condition type
bc = 0 # if bottom boudary condition is given as time series bc = 1
```

Figure 2: *Example of general simulation settings in* `Input.py`.

Table 2: *Description of the general input variables.*

Variable	Description
`site*`	Name of the simulation site. Used for naming output files and directories.
`tstart`	Start date of the simulation period.
`tend`	End date of the simulation period.
`gwli*`	Initial groundwater level relative to the ground surface [m].
`Ref_Time*`	Reference time used for meteorological and groundwater observations.
`GL_to_NAP`	Ground-surface elevation relative to the Dutch reference datum (NAP) [m].
`depths`	Depths defining the soil layers in the subsurface column [m]. Negative values indicate depths below the ground surface.
`types`	Soil type assigned to each layer. Material properties are defined in `Input_soil_material_parameters.py`
`tp_bc`	Type of top boundary condition: Dirichlet (prescribed pore-water pressure) or Neumann (prescribed water flux).
`bc`	Specifies whether a time-varying bottom boundary condition is used. Set `bc = 1` when the bottom boundary condition is provided as a time series.

* : These parameters are required only for SWAP

3.1.2. Soil material parameters

For each soil type, hydraulic and mechanical material parameters must be defined. These parameters are set in the file `Input_soil_material_parameters.py`, as shown in Figure 3 and are listed in Table 3.

```
Soil_parameters ['Clay2_SWP']= {
       'Material_parameters':{
              'gamma_s' : 2.31,
              'gamma_w' : 1,
       },

       'Compression': {
              'kappa' : 0.0868,
              'lambda' : 0.1302,
              'mu' : 0.0028229,
              'Tau_ref': 1,
              'Pg': 400,
              'OCR': 1.5,
              'c' : 10,
              'phi' : 20
       },

       'Water_retention_curve':{
              'vG_WCS' : 0.44,
              'vG_WCR' : 0.00,
              'vG_alpha' : 0.014,
              'vG_n' : 1.12,
              'vG_m' : 0.18,
              'vG_a' : 2000,
              'vG_mod' : 1,
       },

       'Permeability_curve_Perm':{
              'K_tot' : 0.01,
              'K_min' : 0.0001,
              'tau' : 0.5
       }

}
```

Figure 3: *Soil profile in* `input_soil_material_parameters.py`

Table 3: *Soil parameters used in the deformation model*

Category	Variable in code	Parameter	Description
Material Parameters			
	`gamma_s`	γ_s	Unit weight of soil solids [g/cm^3]
	`gamma_w`	γ_w	Unit weight of water [g/cm^3]
Compression Parameters			
	`kappa`	κ	Swelling index (elastic) [–]
	`lambda`	λ	Compression index (plastic) [–]
	`mu`	μ	Creep index [–]]
	`OCR`	OCR	Overconsolidation ratio [–]
	`c`	C	Cohesion [kPa]
	`phi`	ϕ	Internal friction angle [degrees]
Water Retention Curve (van Genuchten)			
	`vG_WCS`	θ_S (vG_{WCS})	Saturated water content [cm^3/cm^3]
	`vG_WCR`	θ_R (vG_{WCR})	Residual water content [cm^3/cm^3]
	`vG_alpha`	vG_α	van Genuchten parameter α [1/cm]
	`vG_n`	vG_n	van Genuchten parameter n [–]
	`vG_m`	vG_m	van Genuchten parameter $m = 1 - 1/n$ [–]
	`vG_a`	vG_a	Air-entry value [cm]
	`vG_mod`	vG_{mod}	van Genuchten model type (modifier flag) [–]
Permeability Curve (Perm Model)			
	`K_tot`	K_{tot}	Hydraulic conductivity at saturation [m/day]
	`K_min`	K_{min}	Minimum hydraulic conductivity at saturation [m/day]
	`tau`	τ	Tortuosity factor or exponent [–]

3.2. Initial and boundary conditions

The initial and boundary conditions required by SubsApp are specified in the `src/input` directory, as shown in Figure 4. These files define the hydraulic state of the soil column at the start of the simulation and the pore-pressure boundary conditions at surface and bottom applied during the simulation period.

```
SubsApp/
        src/
              input/
                    top_pressure/ # Surface boundary condition. Pore pressure or flux
                    bottom_pressure/ # Bottom pore pressure boundary condition
                    pressure_ini/ # Initial pore pressure profile
                    wc_ini/ # Initial volumetric water content profile
```

Figure 4: *Directory structure containing initial and boundary condition files.*

The `bottom_pressure` directory contains time series of pore pressure prescribed at the lower boundaries of the soil column. The `top_pressure` directory can contain either time series of pore pressure or flux at the top boundaries of the soil column. The `pressure_ini` directory contains the initial pore pressure distribution over depth, and `wc_ini` contains the initial volumetric water content profile used to initialize the simulation. The model is

originally developed to use boundary condition derived from SWAP model. The procedure for deriving the boundary conditions from SWAP output is described in Appendix A.

3.3. Model run

Once the input data, initial conditions, and boundary conditions have been specified, SubsApp can be executed. The main script and related source files are located in the `src` directory, as shown in Figure 5.

```
SubsApp/
        src/
               input/ # Initial and boundary conditions
                      main.py # Main deformation-model script
                      build_domain.py # Build soil domain and assign material
                          properties
                      NSDWLs_equations.py # Functions and governing equations
                      NSDWLs_errors.py # Input and geometry consistency checks
                      makefile # Local execution commands
                      run.slurm # SLURM batch script
```

Figure 5: *Directory structure containing the main source code.*

3.3.1. Main script

The script `main.py` is the main code of SubsApp. The model can be executed directly from the command line:

```
python3 main.py
```

Alternatively, the model can be run using the provided `makefile` or `run.slurm` scripts (Section 3.3.2). The model first checks consistency of the model geometry. If any of errors occur, the settings in `Input.py` should be checked and corrected.

The soil column is discretized into elements with a default thickness of 0.05 m. This resolution can be modified.

```
Soil_dimension.precision = 0.05 # m
```

The governing differential equations are solved using the `solve_ivp` function from the SciPy package with the RK23 Runge–Kutta (Virtanen et al. [2020]). The system of equations is defined in:

```
def equations(simulation_time, in_in, sigma_tracker):
```

The maximum time step and relative error tolerance are specified as:

```
max_time_step = 1.0 # day
rtol = 1e-7 # relative tolerance
```

These settings can be adjusted depending on the required accuracy and computational performance.

3.3.2. Makefile and SLURM script

Example `makefile` and `run.slurm` scripts are provided to simplify model execution. These files should be modified according to the user's operating system, Python environment, and HPC cluster configuration. The `makefile`

can be used to run the model and manage output files, while `run.slurm` provides an example SLURM batch script for execution on a computing cluster. The default commands are:

```
make # Run main.py
make clean # Remove simulation results
sbatch run.slurm # Submit a SLURM job
```

Users may need to modify to Python executables, module-loading commands, memory allocation, and partition names to match their local or cluster environment. SubsApp is currently implemented as a one-dimensional single column model and does not include parallel computation. Consequently, requesting multiple CPUs in the SLURM script will generally not reduce computation time for a single simulation. For large ensembles or multiple sites, parallel execution can be achieved by submitting multiple independent jobs.

3.4. Output

Simulation results are saved in the directory `results` as shown in Figure 6:

```
SubsApp/
       results/
             figure/ # .png figure are saved here
             displacement.dat #Surface displacement
             n.dat # porosity over depth
             Se.dat # effective saturation over depth
             theta.dat #volumetric water content over depth
```

Figure 6: *Directory structure containing the results.*

`displacement.dat` contains the simulated time series of surface vertical displacement for the specified simulation period. Displacement is shown in millimetres (mm) and time in days. The files `n.dat`, `Se.dat`, and `theta.dat` contain the simulated porosity (n), effective saturation (S_e), and volumetric water content (θ), respectively. Corresponding figures are generated automatically. In these figures, the horizontal axis represents time, the vertical axis represents depth, and the colour scale represents the value of the simulated variable. For porosity (n) and volumetric water content (θ), the colour scale ranges from the minimum to the maximum value occurring during the simulation. For effective saturation (S_e), the colour scale ranges from 0 to 1, corresponding to completely dry and fully saturated conditions, respectively.

A. SWAP model

The SWAP (Soil-Water-Atmosphere-Plant) model Kroes et al. [2017] is a one-dimensional, physically based simulation tool designed to describe the transport of water, solutes, and heat in variably saturated soils. In the context of the one-dimensional subsidence model, SWAP is used to define the surface pressure boundary condition. In this simulation, only precipitation, evapotranspiration, and groundwater level are used to compute the time series of surface pore pressure. It is important to note a fundamental discrepancy between the assumptions of the SWAP model and those of the vertical deformation model. SWAP does not account for vertical deformation when calculating changes in void ratio and water content over time while the deformation model assumes that changes in void ratio result solely from vertical strain. Due to this inconsistency, the integrated SWAP–deformation model does not capture the physical behavior correctly. Nevertheless, this approach was adopted to assume the general trend of surface pressure. This enables the deformation model to simulate subsidence using precipitation and evapotranspiration as primary inputs, despite the simplification.

A.1. SWAP directory structure

The directory `SubsApp/data/swap` contains the SWAP model executable, post-processing scripts, and example input datasets required to generate the boundary and initial conditions for SubsApp. The directory structure is shown in Figure 7.

```
SubsApp/
 data/
       swap/
              swap420 # Compiled SWAP executable
              input_geology.py # Reads geological settings from input.py
               *.bbc  # Bottom boundary condition (Groundwater level)
               *.YYY  # Meteorological forcing data
              result.* # SWAP output files
              output_swap # Conversion of SWAP output to SubsApp input
                     pressure_bc.py # Generate top boundary pore-pressure time series
                     pressure_ini.py # Generate initial pore-pressure profile
                     water_content.py # Generate initial water-content profile
                     makefile # Run conversion scripts
                     fig/ # Generated figures
```

Figure 7: *Directory structure of the SWAP model and output-processing scripts.*

A.2. Input data

The soil profile, hydraulic properties, and geometry of the soil column used by SWAP are specified in the file `swap.swp`. To ensure consistency between SWAP and SubsApp, the script `input_geology.py` reads the required information from `Input.py` and `Input_soil_material_parameters.py` and writes the corresponding entries to `swap.swp`. The SWAP input file can be generated by executing:

```
python3 input_geology.py
```

After execution, the file `swap.swp` is updated with the soil-layer configuration and material properties defined for the SubsApp simulation.

A.2.1. Groundwater level

Groundwater-level observations are provided through a `*.bbc` file. To use groundwater level as the lower boundary condition, the SWAP option

```
SWBOTB = 1
```

must be specified in `swap.swp`.

The `*.bbc` file contains a time series of groundwater levels with dates in the format `YYYY-MM-DD` and groundwater levels in centimetres (cm). An example is shown below:

```
2024-01-01 -150
2024-01-02 -148
2024-01-03 -147
```

where the second column represents the groundwater level relative to the specified vertical reference datum.

```
* specify DATE1 [YYYY-MM-DD] and GWLEVEL [cm, -10000..1000, R]:
SWBOTB = 1
DATE1 GWLEVEL
2022-07-01 -156.5
2022-07-02 -156.9
2022-07-03 -156.5
2022-07-04 -150.4
```

Figure 8: *Example of groundwater level data in* `*.bbc`

A.2.2. Meteorological condition

Meteorological condition is set as `*.YYY`. This requires the data specified in the 1st and 2nd column in Figure 4. In the Netherlands, where SubsApp is developed, meteorological data from KNMI (`https://www.knmi.nl/nederland-nu/klimatologie/daggegevens`) can be refered in the simulation. To apply this data, the unit and order of the parameteres are converted as shown in Table 4.

Table 4: *Mapping of SWAP input parameters from KNMI data*

SWAP Param	Description	Unit	Format	KNMI Code	KNMI Unit
Station	Station number	–	number	–	–
DD	Day	–	integer	YYYYMMDD	–
MM	Month	–	integer	YYYYMMDD	–
YYYY	Year	–	integer	YYYYMMDD	–
RAD	Global radiation	kJ/m^2	1 decimal	Q	J/cm^2
Tmin	Min. temperature	Celsius	1 decimal	TN	0.1 °C
Tmax	Max. temperature	Celsius	1 decimal	TX	0.1°C
HUM	Humidity	kPa	2 decimal	TG / UG	0.1 °C / %
WIND	Wind speed	m/s	1 decimal	FG	0.1 m/s
RAIN	Daily precipitation	mm	1 decimal	RH	0.1 mm
RETref	Reference evaporation	mm	1 decimal	EV24	0.1 mm
WET	Duration of precipitation per day	days	4 decimal	DR	0.1 hour

A.3. SWAP model run

The orignal source code can be downloaded from https://www.swap.alterra.nl/. In this repository, executable file of swap version 4.2.0 are saved.

```
./swap420
```

A.4. Conversion of SWAP output to SubsApp input

After a SWAP simulation has been completed, the results are written to a set of output files (`result.*`). For SubsApp, only the pore-pressure and water-content information is required. Therefore, the conversion scripts use the file `result.vap`, which contains the simulated hydraulic state of the soil profile.

The following quantities are extracted from the SWAP output and converted into SubsApp input files:

- Initial pore-pressure profile,
- Initial volumetric water-content profile,
- Time series of pore pressure at the top boundary,
- Time series of pore pressure at the bottom boundary.

To generate these files, navigate to the `output_swap` directory and execute:

```
cd output_swap

python3 pressure_bc.py
python3 pressure_ini.py
python3 water_content.py
```

These scripts process the SWAP output and create the initial-condition and boundary-condition files required by SubsApp. The generated files can then be copied to the corresponding input directories described in Section 3.1.

References

C. Belmans, N. De Ridder, and J.C. van Dam. Modeling soil water balance components for a field soil. *Agricultural Water Management*, 6(1-3):21–36, 1983. doi: 10.1016/0378-3774(83)90018-7.

R. Haverkamp, M. Vauclin, J. Touma, P.J. Wierenga, and G. Vachaud. A comparison of numerical simulation models for one-dimensional infiltration. *Soil Science Society of America Journal*, 41(2):285–294, 1977. doi: 10.2136/sssaj1977.03615995004100020011x.

Daniel Hillel. *Introduction to Environmental Soil Physics*. Academic Press, Amsterdam, The Netherlands, 2004. ISBN 9780123485250.

J.G. Kroes, J.C. van Dam, R.P. Bartholomeus, P. Groenendijk, M. Heinen, R.F.A. Hendriks, H.M. Mulder, I. Supit, and P.E.V. van Walsum. SWAP version 4.0: Theory, Description and User Manual. Technical Report 2780, Wageningen Environmental Research, 2017.

Y. Mualem. A new model for predicting the hydraulic conductivity of unsaturated porous media. *Water Resources Research*, 12(3):513–522, 1976. doi: 10.1029/WR012i003p00513.

M. Th. van Genuchten. A closed-form equation for predicting the hydraulic conductivity of unsaturated soils. *Soil Science Society of America Journal*, 44(5):892–898, 1980. doi: 10.2136/sssaj1980.03615995004400050002x.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.